
Quantitative & Machine-Learning Models

The complete mathematical specification

This volume is the exhaustive mathematical reference for every quantitative and machine-learning model in AI Analyst. It specifies the cross-sectional gradient-boosted alpha model, the factor-structured risk models, the dual portfolio optimizers, the walk-forward out-of-sample backtest with Fama–French benchmarking, the rolling robust factor regression, the macro-cycle regime models (PCA/DFM, Gaussian HMM, temporal multimodal VAE, and regime-conditioned factor IC), and the deterministic valuation and scoring models — each with its objective, estimator, algorithm, and the exact warehouse tables and code paths it runs on.

Companion volumes: System Architecture & Operations, qlib Integration, and SEC & EDINET Metrics Mapping & Reconciliation.

Contents

Part I Foundations	3
1 Scope and notation	3
2 The data substrate: a point-in-time fundamentals panel	3
2.1 Point-in-time alignment	3
2.2 Feature families and cross-sectional standardization	3
2.3 Label	4
Part II The qlib quant desk	4
3 Cross-sectional alpha model (return prediction)	4
3.1 Objective	4
3.2 Gradient-boosted decision trees	4
3.3 Training, splitting, and persistence	5
3.4 Evaluation: the information coefficient	5
4 Factor-structured risk models (forward covariance)	5
4.1 Statistical factor covariance (qlib StructuredCovEstimator)	5
4.2 Ledoit–Wolf shrinkage (constant-correlation target)	6
4.3 Fama–French fundamental covariance	6
5 Portfolio optimization (dual backend)	6
5.1 Native optimizer: multi-term mean–variance utility	6
5.2 qlib optimizers (long-only, fully invested)	7
5.3 Enhanced indexing (benchmark-relative)	7
5.4 Reported portfolio statistics	7
6 Walk-forward out-of-sample backtest	7
6.1 Walk-forward predictions	7
6.2 Portfolio construction	8
6.3 Performance and risk metrics	8
6.4 Fama–French benchmarking and factor attribution	8
Part III Factor and macro-regime models	8
7 Rolling Fama–French factor regression	8
7.1 Robust estimation (Huber IRLS)	9
7.2 HAC (Newey–West) standard errors	9
7.3 Stored diagnostics and derived series	9
8 Macro-cycle regime models	9
8.1 Feature construction	9
8.2 Baseline: PCA / dynamic-factor compression	9
8.3 Phase calibration and labeling	9
8.4 Gaussian Hidden Markov Model	10
8.5 Temporal multimodal variational autoencoder	10
8.6 Regime-conditioned factor IC	10
Part IV Deterministic valuation and scoring	11
9 Valuation models	11
9.1 Discounted cash flow (unlevered FCFF)	11
9.2 Cost of capital (WACC)	11
9.3 Reverse DCF, scenarios, sum-of-the-parts, triangulation	12
10 Scoring and ranking models	12
10.1 Value + sentiment interest score	12

10.2 Group relative-value composite	12
10.3 MD&A management tone	12
Part V How the models connect	12
11 The model map	12

Part I Foundations

1 Scope and notation

AI Analyst answers two different questions with two different families of model. The *valuation* models (Part IV) answer “*what is this business worth?*” deterministically, from projected cash flows and a cost of capital. The *quantitative* models (Parts II–III) answer “*what should I expect from this stock, and how should I size it?*” statistically, from a learned cross-sectional return forecast, a forward covariance, and a portfolio optimizer, conditioned on an estimated macro regime. This document is the mathematical specification of both families.

Notation. $\mathbf{w} \in \mathbb{R}^N$ portfolio weights; $\boldsymbol{\mu} \in \mathbb{R}^N$ expected (annualized) returns; $\Sigma \in \mathbb{R}^{N \times N}$ return covariance; r_f the risk-free rate; $\mathbf{1}$ the ones vector; $\odot$ the elementwise (Hadamard) product; z a cross-sectional z -score. Time subscripts t index months unless stated otherwise. Annualization uses 252 trading days for daily series and 12 for monthly series. A “hat” ($\hat{\cdot}$) denotes an estimate or a model prediction.

Every model is designed to *degrade gracefully*: if a trained artifact, an optional dependency (`hmmlearn`, `PyTorch`, a MIP solver), or a warehouse table is absent, the code returns a defined fallback rather than failing, so the application always runs. Those fallbacks are stated with each model.

2 The data substrate: a point-in-time fundamentals panel

Every cross-sectional model consumes the same object: a monthly panel of standardized fundamentals aligned *point-in-time* (PIT) so that no signal ever uses data that was not yet public.

2.1 Point-in-time alignment

A fundamental value with fiscal-period end p is stamped usable only at

$$\tau(p) = \text{monthend}(p + 90 \text{ days}), \quad (1)$$

a conservative proxy for the filing lag. In SQL the alignment is `(date_trunc('month', period_end + INTERVAL '90 days') + INTERVAL '1 month - 1 day')`. The alpha model (§3) and the regime-IC engine (§8.6) share this one loader, so the research signal and the production forecast see an identical, lookahead-safe history.

2.2 Feature families and cross-sectional standardization

Features are importance-ranked standardized fundamentals (`fact_metrics_{us,jp}`, keeping importance ≤ 2) drawn from four families:

Family	Representative factors (substring patterns)
value	book-to-market, earnings yield, FCF yield, dividend, EV/·, valuation multiples
quality	return-on-·, ROE/ROA, margins, accruals, cash conversion, asset turnover
growth	revenue/earnings growth, investment, capex, R&D, sales change
market_factor	market beta, volatility, momentum, residual/abnormal return

For each factor j , name i , and month t , the raw value $x_{i,t}^{(j)}$ is *cross-sectionally* standardized and winsorized (mirroring qlib’s `CSZScoreNorm`):

$$z_{i,t}^{(j)} = \text{clip}\left(\frac{x_{i,t}^{(j)} - \bar{\mu}_t^{(j)}}{\hat{\sigma}_t^{(j)}}, -3, 3\right), \quad \bar{\mu}_t^{(j)} = \frac{1}{N_t} \sum_i x_{i,t}^{(j)}, \quad \hat{\sigma}_t^{(j)} = \text{std}_i x_{i,t}^{(j)}. \quad (2)$$

Standardizing within each date makes factors comparable across names and dates and robust to per-metric scale; a zero-variance cross-section maps to 0 (the neutral value). A missing feature is imputed to the cross-sectional mean, i.e. $z = 0$.

2.3 Label

The supervised label is the forward one-month (or three-month) total return,

$$y_{i,t} = r_{i,t \rightarrow t+h}, \quad h \in \{1, 3\} \text{ months.} \quad (3)$$

The panel is built as a qlib MultiIndex frame — rows (datetime, instrument), columns (feature | label) — consumed directly by `DataHandlerLP.from_df`. JP fundamentals carry a .T suffix that is stripped so fundamentals join prices. Thin cross-sections (< 30 names) are dropped as unstable for ranking and normalization.

Design

The panel is the single contract between the warehouse and the models: a monthly, cross-sectional, lookahead-safe frame of z -scored fundamentals plus a forward return. Expected returns from the alpha model are therefore *monthly* forecasts — annualize by $\times 12$ before feeding a portfolio optimizer.

Part II The qlib quant desk

3 Cross-sectional alpha model (return prediction)

Code: `api/quant/qlib_alpha.py`, `api/quant/qlib_data.py`, `api/quant/alpha_signal.py`; built on qlib's `LGBModel` (LightGBM).

3.1 Objective

Learn a function $f: \mathbf{z} \mapsto \hat{y}$ mapping the standardized cross-sectional feature vector of a name to its expected forward return. The prediction $\hat{y} = f(\mathbf{z})$ is the *alpha signal*: a per-name, per-month score interpreted as an expected forward return.

3.2 Gradient-boosted decision trees

The model is an additive ensemble of M regression trees with learning rate η ,

$$F_M(\mathbf{z}) = \sum_{m=1}^M \eta h_m(\mathbf{z}), \quad h_m \text{ a regression tree.} \quad (4)$$

Under the squared-error objective $\mathcal{L} = \sum_i \frac{1}{2} (y_i - F(\mathbf{z}_i))^2$, boosting stage m fits the new tree h_m to the negative functional gradient — here simply the residual — with unit Hessian:

$$g_i = -\left. \frac{\partial \mathcal{L}}{\partial F} \right|_{F_{m-1}} = y_i - F_{m-1}(\mathbf{z}_i), \quad h_i = \frac{\partial^2 \mathcal{L}}{\partial F^2} = 1. \quad (5)$$

LightGBM grows each tree *leaf-wise* over a histogram approximation of the features. Splitting a node with instance set I into I_L, I_R yields the gain

$$\text{Gain} = \frac{1}{2} \left[\frac{(\sum_{i \in I_L} g_i)^2}{\sum_{i \in I_L} h_i + \lambda} + \frac{(\sum_{i \in I_R} g_i)^2}{\sum_{i \in I_R} h_i + \lambda} - \frac{(\sum_{i \in I} g_i)^2}{\sum_{i \in I} h_i + \lambda} \right] - \gamma, \quad (6)$$

where λ is the L_2 leaf regularizer and γ the per-split penalty. Trees capture non-linearities and factor *interactions* that a linear factor model cannot represent.

Shipped hyperparameters (DEFAULT_LGB_PARAMS)

<code>loss = mse</code>	<code>learning_rate = 0.03</code>	<code>num_leaves = 31</code>	<code>max_depth = 6</code>
<code>colsample_bytree = 0.8</code>	<code>subsample = 0.8</code>	<code>subsample_freq = 1</code>	<code>min_child_samples = 50</code>
<code>lambda_l1 = 1.0</code>	<code>lambda_l2 = 1.0</code>	<code>num_boost_round = 500</code>	<code>early_stopping_rounds = 40</code>

Regularization (L_1/L_2 leaf penalties, shallow depth, feature/row subsampling) plus early stopping on a held-out validation segment control overfitting in a setting with a low signal-to-noise ratio.

3.3 Training, splitting, and persistence

The panel is split *chronologically* by unique dates into contiguous train / validation / test segments (defaults 70/15/15). Training runs the real `qlib DatasetH` pipeline; the `AlphaLGB` subclass overrides `fit` to attach LightGBM's early-stopping and evaluation callbacks directly, so training runs with plain LightGBM and needs no `qlib.init` or metric-tracking backend. Prediction runs the stored booster on any reindexed feature matrix — including a single live month — so scoring never rebuilds a `DatasetH`. Artifacts (the booster plus metadata: feature identity/order, label horizon, train range, IC) are serialized per jurisdiction and hot-reloaded on file `mtime`.

For the live cross-section the model builds a *features-only* panel (`require_label=False`) over a short lookback and scores the newest month: the current month has no realized forward return, so a label join would drop exactly the cross-section we want to score.

3.4 Evaluation: the information coefficient

Predictive power is measured per month by the cross-sectional correlation between prediction and realized return:

$$IC_t = \text{Corr}(\hat{y}_{\cdot,t}, y_{\cdot,t}) \text{ (Pearson)}, \quad \text{RankIC}_t = \rho_{\text{Spearman}}(\hat{y}_{\cdot,t}, y_{\cdot,t}), \quad (7)$$

summarized by the mean IC and the information ratio of the signal, $\text{ICIR} = \overline{\text{IC}} / \text{std}(\text{IC})$ (and its rank analogue). A monthly equity signal with $\overline{\text{RankIC}} \approx 0.05\text{--}0.10$ is usable. The *in-sample* test segment of the shipped US/JP models reaches $\overline{\text{RankIC}} \approx 0.08\text{--}0.14$; the *out-of-sample* walk-forward figure (§6) is materially lower and is the number used for any performance claim.

4 Factor-structured risk models (forward covariance)

Code: `api/quant/qlib_risk.py` (statistical) and `api/quant/risk.py` (Ledoit–Wolf, Fama–French).

A portfolio optimizer needs a forward covariance Σ that is well conditioned and invertible. A raw $N \times N$ sample covariance from a short daily window is noisy and near-singular. All three risk models replace it with a structured, lower-parameter estimator. Each is selectable at runtime.

4.1 Statistical factor covariance (qlib StructuredCovEstimator)

Returns are modeled as driven by K latent statistical factors,

$$X = B F^T + U, \quad \text{Cov}(X) = F \text{Cov}(B) F^T + \text{diag}(\text{Var}(U)), \quad (8)$$

where $X \in \mathbb{R}^{T \times N}$ is the return panel, $F \in \mathbb{R}^{N \times K}$ the factor *exposures* (principal components; factor analysis is also supported), $B \in \mathbb{R}^{T \times K}$ the factor *returns* (component scores), and U the idiosyncratic residual. Writing $\Sigma_B = \text{Cov}(B)$ and $\mathbf{d} = \text{Var}(U)$,

$$\Sigma = F \Sigma_B F^T + \text{diag}(\mathbf{d}) \quad (9)$$

— a low-rank-plus-diagonal shrinkage of the sample covariance. The number of factors is clamped to $K = \min(K_0, N - 1, T - 2)$ for safety. Estimation is on *raw* returns (the estimator's percentage-rescaling is disabled, keeping variances in return units); Σ_B and $\mathbf{d}$ are annualized by $\times 252$, and Σ

is rebuilt from the annualized decomposition, symmetrized, and given a 10^{-10} ridge for numerical positive-definiteness. Forward volatility per name is $\sigma_i = \sqrt{\Sigma_{ii}}$. The decomposition $(F, \Sigma_B, \mathbf{d})$ is consumed directly by the enhanced-indexing optimizer (§5.3).

4.2 Ledoit–Wolf shrinkage (constant-correlation target)

An analytic shrinkage of the MLE sample covariance S toward a constant-correlation target F . With variances s_{ii} , standard deviations $\sigma_i = \sqrt{s_{ii}}$, and mean pairwise correlation $\bar{r} = \frac{2}{N(N-1)} \sum_{i < j} s_{ij} / (\sigma_i \sigma_j)$, the target is $F_{ii} = s_{ii}$, $F_{ij} = \bar{r} \sigma_i \sigma_j$, and

$$\Sigma^* = \delta F + (1 - \delta)S, \quad \delta = \text{clip}\left(\frac{\pi/\gamma}{T}, 0, 1\right), \quad (10)$$

where $\gamma = \|S - F\|_F^2$ measures target misspecification and $\pi = \sum_{i,j} [\frac{1}{T} \sum_t (x_{ti}x_{tj} - s_{ij})^2]$ is the summed asymptotic variance of the entries of S . The optimal intensity δ trades sample noise against target bias; the result is annualized $\times 252$.

4.3 Fama–French fundamental covariance

A named-factor analogue of the statistical model:

$$\Sigma = B F_{\text{ann}} B^\top + D, \quad (11)$$

with B the per-name FF loadings (`fact_factor_loadings`, §7), $F_{\text{ann}} = 252 \cdot \text{Cov}(\text{factor returns})$ estimated over the lookback, and $D = \text{diag}(\text{residual_vol}_i^2)$ from the regression meta. Model ids `ff3`, `ff5`, `ff5_plus_mom` select the factor set (FF6 = FF5 + momentum). Names with missing loadings fall back to their sample diagonal with a surfaced warning; an unavailable factor model degrades to Ledoit–Wolf.

5 Portfolio optimization (dual backend)

Code: `api/quant/qlib_optimize.py` dispatches at runtime to a *native* optimizer (`api/quant/optimize.py`) or any qlib optimizer; both consume the same μ, Σ and return a uniform `PortfolioSolution`.

5.1 Native optimizer: multi-term mean–variance utility

The native backend maximizes a mean–variance utility with factor-tilt, turnover, and concentration penalties:

$$\max_{\mathbf{w}} \quad \mathbf{w}^\top \mu - \frac{1}{2} \lambda \mathbf{w}^\top \Sigma \mathbf{w} - \gamma_f \|S \odot a \odot (B^\top \mathbf{w} - \mathbf{t})\|_2^2 - \gamma_{\text{to}} \|\mathbf{w} - \mathbf{w}_0\|_1 - \gamma_c \|\mathbf{w}\|_2^2, \quad (12)$$

subject to full investment $\mathbf{1}^\top \mathbf{w} = 1$, per-name bounds $l_i \leq w_i \leq u_i$, a gross cap $\sum_i |w_i| \leq G$, a short-leg cap $\sum_i \max(-w_i, 0) \leq \rho \sum_i |w_i|$, sector caps $\mathbf{a}_s^\top \mathbf{w} \leq c_s$, an annualized volatility cap $\mathbf{w}^\top \Sigma \mathbf{w} \leq \sigma_{\text{max}}^2$, and factor caps/ranges $|\mathbf{b}_j^\top \mathbf{w}| \leq \text{cap}_j$. Here λ is risk aversion; γ_f penalizes deviation of the portfolio’s factor exposures $B^\top \mathbf{w}$ from targets $\mathbf{t}$ (scaled by S , gated by the active mask a); γ_{to} penalizes turnover against the current book $\mathbf{w}_0$; and γ_c penalizes concentration.

The problem is solved by sequential least-squares programming (SLSQP), which handles the nonlinear volatility and factor constraints natively and converges quickly at portfolio scale ($N \lesssim 100$). SLSQP is given the analytic gradient

$$\nabla = -\mu + \lambda \Sigma \mathbf{w} + 2\gamma_f B(S^2 \odot a \odot (B^\top \mathbf{w} - \mathbf{t})) + \gamma_{\text{to}} \text{sign}(\mathbf{w} - \mathbf{w}_0) + 2\gamma_c \mathbf{w}. \quad (13)$$

Cardinality (mixed-integer program). When a maximum position count $\|\mathbf{w}\|_0 \leq K$ is requested, the problem becomes a mixed-integer program with binary inclusion indicators $y_i \in \{0, 1\}$:

$$\sum_i y_i \leq K, \quad \begin{cases} 0 \leq w_i \leq u_i y_i, & \text{long-only,} \\ -u_i y_i \leq w_i \leq u_i y_i, & \text{long/short,} \end{cases} \quad w_i \geq f y_i \text{ (floor on included names),} \quad (14)$$

retaining the convex volatility, sector, and factor constraints. It is solved via `cvxpy` over the first available MIP backend (HiGHS $\rightarrow$ SCIP $\rightarrow$ CBC $\rightarrow$ GLPK_MI $\rightarrow$ MOSEK); if none is installed the route surfaces a clear “MIP unavailable” error rather than silently dropping the constraint.

Diagnostics and frontier. The solver reports factor exposures $B^T \mathbf{w}$, marginal risk contributions $(\Sigma \mathbf{w}) \odot \mathbf{w} / \sqrt{\mathbf{w}^T \Sigma \mathbf{w}}$, the effective number of names $1 / \sum_i w_i^2$, and the covariance condition number; it also sweeps λ over a fixed grid to trace a mini efficient frontier.

5.2 qlib optimizers (long-only, fully invested)

All satisfy $\mathbf{1}^T \mathbf{w} = 1$, $\mathbf{w} \geq 0$:

Backend	Objective
GMV (min variance)	$\min_{\mathbf{w}} \mathbf{w}^T \Sigma \mathbf{w}$
MVO (mean–variance)	$\max_{\mathbf{w}} \lambda \tilde{r}^T \mathbf{w} - \mathbf{w}^T \Sigma \mathbf{w} \quad (\tilde{r} \text{ scaled to } \Sigma\text{'s volatility})$
RP (risk parity)	choose $\mathbf{w}$ s.t. $w_i (\Sigma \mathbf{w})_i = w_j (\Sigma \mathbf{w})_j \quad \forall i, j$
Inverse-vol	$w_i = \frac{1/\sigma_i}{\sum_j 1/\sigma_j}, \quad \sigma_i = \sqrt{\Sigma_{ii}}$

Risk parity equalizes the risk contributions $RC_i = w_i (\Sigma \mathbf{w})_i$, which sum to total variance $\sum_i RC_i = \mathbf{w}^T \Sigma \mathbf{w}$; GMV/MVO/RP are solved with `scipy.optimize` and admit optional turnover (δ) and L_2 (α) terms.

5.3 Enhanced indexing (benchmark-relative)

Given benchmark weights $\mathbf{w}_b$, active weights $\mathbf{d} = \mathbf{w} - \mathbf{w}_b$, and active factor exposure $\mathbf{v} = \mathbf{d}^T F$, the enhanced-indexing optimizer maximizes active return minus active (tracking-error) risk under bounded name and factor deviations:

$$\max_{\mathbf{w}} \mathbf{d}^T \mathbf{r} - \lambda (\mathbf{v}^T \Sigma_B \mathbf{v} + \mathbf{d}^T \text{diag}(\mathbf{d}_u) \mathbf{d}) \quad \text{s.t.} \quad \mathbf{w} \geq 0, \mathbf{1}^T \mathbf{w} = 1, \|\mathbf{w} - \mathbf{w}_0\|_1 \leq \delta, |\mathbf{d}| \leq b_{\text{dev}}, |\mathbf{v}| \leq f_{\text{dev}}, \quad (15)$$

where $\mathbf{d}_u = \text{Var}(U)$ is the specific variance. It is a convex quadratic program consuming the statistical-covariance decomposition $(F, \Sigma_B, \mathbf{d}_u)$ of §4 directly, solved by `cvxpy/ECOS`; on solver failure it degrades to the benchmark weights with a warning.

5.4 Reported portfolio statistics

For a solution $\mathbf{w}^*$ the desk reports $\mathbb{E}[r]_{\text{ann}} = \mathbf{w}^{*T} \boldsymbol{\mu}$, $\sigma_{\text{ann}} = \sqrt{\mathbf{w}^{*T} \Sigma \mathbf{w}^*}$, Sharpe = $(\mathbb{E}[r]_{\text{ann}} - r_f) / \sigma_{\text{ann}}$, and the factor exposures and risk contributions above.

6 Walk-forward out-of-sample backtest

Code: `api/quant/qlib_backtest.py`. A cross-sectional signal backtest that is out-of-sample by construction, benchmarked against Fama–French.

6.1 Walk-forward predictions

Let $t_0 < t_1 < \dots < t_T$ be the panel months, restricted to an investable universe (market cap $\geq$ \$2B) with each month’s forward returns winsorized to the 1st/99th cross-sectional percentile (removing micro-cap data artifacts). For each test month t_i with $i \geq m_{\text{min}}$ (default $m_{\text{min}} = 12$; refit every $q = 3$ months) a booster g_{t_i} is trained on *all prior* months $\{t_j : t_j < t_i\}$ and predicts $\hat{\alpha}_{k,t_i} = g_{t_i}(\mathbf{z}_{k,t_i})$. No month is ever scored by a model that trained on it. The expensive walk-forward predictions are cached separately from portfolio construction, so re-tuning top- k or long/short re-ranks instantly.

6.2 Portfolio construction

Ranking names by $\hat{\alpha}$ each month, the equal-weight books realize

$$r_t^L = \frac{1}{k} \sum_{i \in \text{top-}k} r_{i,t}, \quad r_t^{\text{LS}} = \frac{1}{k} \sum_{i \in \text{top-}k} r_{i,t} - \frac{1}{k} \sum_{i \in \text{bot-}k} r_{i,t}, \quad (16)$$

for the long-only and long/short strategies respectively; a month is skipped if it has fewer than $\max(k, 10)$ (long) or $2k$ (long/short) investable names.

6.3 Performance and risk metrics

With monthly returns $\{r_t\}_{t=1}^n$, equity $E_t = \prod_{s \leq t} (1 + r_s)$, and monthly risk-free $r_{f,t}$:

$$R_{\text{ann}} = \left(\prod_t (1 + r_t) \right)^{12/n} - 1, \quad \sigma_{\text{ann}} = \text{std}(r_t) \sqrt{12}, \quad (17)$$

$$\text{Sharpe} = \frac{R_{\text{ann}} - R_{\text{ann}}^f}{\sigma_{\text{ann}}}, \quad \text{Sortino} = \frac{R_{\text{ann}} - R_{\text{ann}}^f}{\text{std}(r_t : r_t < 0) \sqrt{12}}, \quad (18)$$

$$\text{MaxDD} = \min_t \left(\frac{E_t}{\max_{s \leq t} E_s} - 1 \right), \quad \text{Hit} = \frac{1}{n} \sum_t \mathbf{1}[r_t > 0]. \quad (19)$$

6.4 Fama–French benchmarking and factor attribution

The benchmark is the FF *market* total return $r_t^m = (\text{Mkt-RF})_t + (\text{RF})_t$, compounded from daily `fact_fama_french` to monthly and aligned to $\{r_t\}$. Relative to it,

$$\text{excess} = R_{\text{ann}} - R_{\text{ann}}^m, \quad \text{TE} = \text{std}(r_t - r_t^m) \sqrt{12}, \quad \text{IR} = \frac{\text{excess}}{\text{TE}}, \quad \beta = \frac{\text{Cov}(r, r^m)}{\text{Var}(r^m)}. \quad (20)$$

To separate genuine alpha from factor exposure, the strategy's excess return is regressed on the FF five factors plus momentum,

$$r_t - r_{f,t} = \alpha + \beta_{\text{Mkt}} f_t^{\text{Mkt}} + \beta_{\text{SMB}} f_t^{\text{SMB}} + \beta_{\text{HML}} f_t^{\text{HML}} + \beta_{\text{RMW}} f_t^{\text{RMW}} + \beta_{\text{CMA}} f_t^{\text{CMA}} + \beta_{\text{Mom}} f_t^{\text{Mom}} + \varepsilon_t. \quad (21)$$

OLS on the design $A = [\mathbf{1} \ X]$ gives $\hat{\theta} = (A^\top A)^{-1} A^\top y$, residual variance $\hat{s}^2 = \|\varepsilon\|^2 / (n - K - 1)$, coefficient covariance $\hat{s}^2 (A^\top A)^{-1}$, the alpha t -statistic $t_\alpha = \hat{\alpha} / \text{se}(\hat{\alpha})$, annualized alpha $\alpha_{\text{ann}} = (1 + \hat{\alpha})^{12} - 1$, and $R^2 = 1 - \|\varepsilon\|^2 / \sum_t (y_t - \bar{y})^2$.

Interpretation

The walk-forward design is what makes the backtest honest: because every month is scored by a model that never saw it, the out-of-sample `RankIC` and t_α are the figures used for any claim about the signal. They are markedly weaker than the in-sample test-segment numbers of §3 — the expected outcome of removing look-ahead, and the reason performance is always quoted from this path.

Part III Factor and macro-regime models

7 Rolling Fama–French factor regression

Code: `xbr1_sec/sec/sources/factor_model.py`. Per ticker, over a rolling 252-day window (stepped every 21 trading days, minimum 126 observations), excess returns are regressed on the FF factors of the chosen model (FF3/FF4/FF5/FF6):

$$r_{i,t} - r_{f,t} = \alpha_i + \sum_k \beta_{i,k} f_{k,t} + \varepsilon_{i,t}. \quad (22)$$

Factor series are region-resolved (US, JP, Developed, Emerging) from the available Ken French datasets and converted to decimals; the dependent window is winsorized to the 1st/99th percentile.

7.1 Robust estimation (Huber IRLS)

Rather than plain OLS, coefficients are estimated by iteratively reweighted least squares with Huber weights, which down-weight outlier days. Over a small number of iterations, with scale $\hat{s} = \text{median}(|e|)/0.6745$ and $u = |e|/\hat{s}$,

$$\hat{\beta}^{(k+1)} = (X^\top W^{(k)} X)^{-1} X^\top W^{(k)} y, \quad w_i = \begin{cases} 1, & u_i \leq \delta, \\ \delta/u_i, & u_i > \delta, \end{cases} \quad \delta = 1.345. \quad (23)$$

7.2 HAC (Newey–West) standard errors

Standard errors are heteroskedasticity- and autocorrelation-consistent with Bartlett weights over $L = 5$ lags:

$$\widehat{\text{Var}}(\hat{\beta}) = (X^\top X)^{-1} \hat{S} (X^\top X)^{-1}, \quad \hat{S} = \hat{\Gamma}_0 + \sum_{\ell=1}^L w_\ell (\hat{\Gamma}_\ell + \hat{\Gamma}_\ell^\top), \quad w_\ell = 1 - \frac{\ell}{L+1}, \quad (24)$$

with $\hat{\Gamma}_\ell = \sum_t (x_t \varepsilon_t)(x_{t-\ell} \varepsilon_{t-\ell})^\top$.

7.3 Stored diagnostics and derived series

Each window stores α (annualized $\times 252$), the betas, t -statistics, p -values, and standard errors, plus R^2 , adjusted $R^2 = 1 - (1 - R^2) \frac{n-1}{n-k}$, F -statistic and its p -value, RMSE, residual volatility $\hat{\sigma}_\varepsilon \sqrt{252}$ (the idiosyncratic D of §4.3), the Durbin–Watson statistic $\sum_t (\varepsilon_t - \varepsilon_{t-1})^2 / \sum_t \varepsilon_t^2$, and the design condition number. A composite `quality_score` $\in \{0, \dots, 5\}$ counts how many of five gates a window passes (adj. $R^2 > 0.20$; F $p < 0.05$; $n \geq 200$; condition number < 30 ; $1.5 \leq \text{DW} \leq 2.5$). Between refits, per-day model-implied returns $\hat{r}_{i,t} = \alpha_i/252 + \sum_k \beta_{i,k} f_{k,t} + r_{f,t}$ are written to `fact_factor_implied_returns`. These loadings feed the WACC (§9.2) and the FF structured covariance (§4.3).

8 Macro-cycle regime models

Code: `xbrl_sec/sec/cycle/`. The regime stack has three layers: a multimodal feature set, a dimensionality-reduction *baseline* that produces latent cycle factors, and two *regime models* (HMM and VAE) that run on top, each mapped to interpretable business-cycle phases and then linked to forecasting through regime-conditioned factor IC.

8.1 Feature construction

`cycle/features.py` assembles a monthly, point-in-time, multimodal feature matrix across three modalities — *macro* (growth, inflation, rates, liquidity, credit), *market*, and *fundamental* — each observation stamped with its true availability date. Features are standardized with an *expanding* (walk-forward) mean and standard deviation, $z_{f,t} = (x_{f,t} - \mu_{f,\leq t}) / \sigma_{f,\leq t}$ (minimum 12 observations), so no future information leaks into any month’s normalization.

8.2 Baseline: PCA / dynamic-factor compression

`cycle/baselines.py` compresses the (mean-centered) feature matrix to K latent cycle factors via the SVD $X = U \Sigma V^\top$, taking the leading components factors $= U_{:,1:K} \Sigma_{1:K}$ with explained-variance ratios $\sigma_k^2 / \sum \sigma^2$. The dfm (dynamic-factor) variant additionally smooths the factors with an exponentially weighted moving average (span 3), approximating the persistence of a dynamic factor model. The factors are written to `fact_cycle_state_monthly.latent_cycle` and are the input to the two regime models.

8.3 Phase calibration and labeling

Latent factors have arbitrary sign and order, so `cycle/phase.py` anchors them to interpretable phases. It (i) selects the factor most correlated with an external stress/recession proxy from the macro

store and orients it so “high = stress”; (ii) maps the factor to a rolling stress *percentile* s_t over a 120-month window; and (iii) thresholds s_t into four phases {contraction, late_cycle, mid_expansion, early_expansion}. For US models the thresholds are calibrated against the NBER recession chronology (via USREC’s trough convention) by maximizing balanced accuracy $\frac{1}{2}(\text{TPR} + \text{TNR})$ over a grid; labels are then smoothed (EWMA) and passed through a minimum-phase-duration filter that dissolves runs shorter than three months. Phase confidence is a bounded function of distance from the 0.5 percentile, $c_t = \text{clip}(1.5 |s_t - 0.5| + 0.45, 0.45, 0.90)$.

8.4 Gaussian Hidden Markov Model

`cycle/hmm.py` fits a Gaussian HMM (default $K = 4$ states, full covariance) over the latent cycle factors $\mathbf{x}_t$:

$$\mathbf{x}_t \mid s_t=k \sim \mathcal{N}(\boldsymbol{\mu}_k, \Sigma_k), \quad A_{ij} = \mathbb{P}(s_t=j \mid s_{t-1}=i), \quad \pi_k = \mathbb{P}(s_1=k). \quad (25)$$

The parameters $\{\pi, A, \boldsymbol{\mu}_k, \Sigma_k\}$ are fit by Baum–Welch (the EM algorithm for HMMs). The forward–backward recursions define $a_t(k) = \mathbb{P}(\mathbf{x}_{1:t}, s_t=k)$ and $b_t(k) = \mathbb{P}(\mathbf{x}_{t+1:T} \mid s_t=k)$, from which the smoothed state posterior is

$$\gamma_t(k) = \mathbb{P}(s_t=k \mid \mathbf{x}_{1:T}) = \frac{a_t(k) b_t(k)}{\sum_{k'} a_t(k') b_t(k')}. \quad (26)$$

The per-month confidence is $\max_k \gamma_t(k)$ and the MAP state path is recovered by Viterbi. States are mapped to phases by ordering their mean stress score (§8.3). If `hmmlearn` is unavailable, a deterministic k -means-style assignment with empirical transition counts produces the same output shape.

8.5 Temporal multimodal variational autoencoder

`cycle/vae.py` is an optional VAE with encoder $q_\phi(\mathbf{z} \mid \mathbf{x}) = \mathcal{N}(\boldsymbol{\mu}_\phi(\mathbf{x}), \text{diag } \boldsymbol{\sigma}_\phi^2(\mathbf{x}))$, decoder $p_\theta(\mathbf{x} \mid \mathbf{z})$, and prior $p(\mathbf{z}) = \mathcal{N}(\mathbf{0}, I)$. It is trained by maximizing the evidence lower bound with a temporal smoothness penalty:

$$\mathcal{L}(\theta, \phi) = \underbrace{\mathbb{E}_{q_\phi}[\log p_\theta(\mathbf{x} \mid \mathbf{z})]}_{\text{reconstruction}} - \beta \underbrace{\text{KL}(q_\phi(\mathbf{z} \mid \mathbf{x}) \parallel p(\mathbf{z}))}_{\text{regularization}} - \eta \sum_t \|\boldsymbol{\mu}_{\phi,t} - \boldsymbol{\mu}_{\phi,t-1}\|^2, \quad (27)$$

using the reparameterization $\mathbf{z} = \boldsymbol{\mu}_\phi + \boldsymbol{\sigma}_\phi \odot \boldsymbol{\epsilon}$, $\boldsymbol{\epsilon} \sim \mathcal{N}(\mathbf{0}, I)$ (in the shipped configuration $\beta = 0.05$, $\eta = 0.02$). The latent dimensions are split into interpretable *growth* / *inflation* / *rates-liquidity* / *credit-stress* / *market* / *fundamentals* factors. The smoothed reconstruction-error stress percentile is thresholded into phases exactly as in §8.3, and per-modality reconstruction MSE attributes stress to macro vs. market vs. fundamentals. If PyTorch is absent, a PCA surrogate produces the same persisted output shape.

8.6 Regime-conditioned factor IC

`cycle/ic.py` is the link from the regime model to forecasting. For each fundamental factor m , *conditioned on the regime*, it computes the rank information coefficient against forward returns over each month and horizon:

$$\text{IC}_{\text{regime}}(m) = \rho_{\text{Spearman}}(x_{m,i}, r_i^{\text{fwd}} \mid \text{regime}). \quad (28)$$

Two variants are stored to `fact_equity_factor_ic_regime`: a *hard-assigned* IC (grouping by the MAP phase label) and a *probability-weighted* IC that weights each observation by the regime posterior $\gamma_t(k)$ (soft assignment), using the weighted Spearman correlation of the value and forward-return ranks. The result answers *which factors have actually predicted returns in conditions like today’s*, and drives the scanner’s factor-family weights (§10.1).

From regime to score: the one live consumption path

`ic_weights.family_weights` reads the *latest date's* regime-IC rows, aggregates per-metric $|IC|$ to family-level emphasis $e_{fam} = \overline{|IC|}_{fam}$ (normalized to sum to one over value / growth / quality), and caches it. The scanner then re-splits its fundamental score budget in proportion to e_{fam} : the interest score leans harder on whichever factor family the current regime says is most predictive, and reverts *exactly* to the fixed weights when the IC table is unavailable. The learned alpha model itself is *regime-agnostic* — it learns from factor values, and the regime enters forecasting through the score weights, not (currently) as a model feature.

Part IV Deterministic valuation and scoring

9 Valuation models

Code: `ai_analyst/dcf_engine.py`, `committee/wacc.py`, `committee/valuation.py`, `committee/segments.py`. Deterministic, on a 7-year explicit horizon (DCF_HORIZON_YEARS). The models are intentionally transparent so the equity-research “why” of every number reads off the projected statements; the LLM committee supplies only the bounded assumptions, and the math below turns them into figures.

9.1 Discounted cash flow (unlevered FCFF)

For each forecast year $i = 1, \dots, 7$:

$$\text{Rev}_i = \text{Rev}_{i-1}(1 + g_i), \quad \text{EBIT}_i = \text{Rev}_i \cdot m, \quad (29)$$

$$\text{NOPAT}_i = \text{EBIT}_i - \max(\text{EBIT}_i, 0) \tau, \quad \text{FCF}_i = \text{NOPAT}_i + \text{D\&A}_i - \text{Capex}_i - \Delta \text{NWC}_i, \quad (30)$$

with D&A, Capex, and ΔNWC taken as fractions of revenue (D&A% from the latest historical ratio). Terminal value uses Gordon growth, and the enterprise/equity bridge is

$$\text{TV} = \frac{\text{FCF}_7(1 + g_\infty)}{\text{WACC} - g_\infty}, \quad \text{EV} = \sum_{i=1}^7 \frac{\text{FCF}_i}{(1 + \text{WACC})^i} + \frac{\text{TV}}{(1 + \text{WACC})^7}, \quad (31)$$

Equity = EV – NetDebt, per share = Equity/shares. A 5×5 sensitivity grid varies WACC by ± 200 bp and g_∞ by ± 100 bp (a growth $\times$ WACC grid is also produced around the base case).

9.2 Cost of capital (WACC)

Cost of equity is built from the FF factor betas (§7) and long-run premia $\Pi_k = 252 \cdot \bar{f}_k$:

$$R_e^{\text{CAPM}} = r_f + \beta_{\text{Mkt}} \text{ERP}, \quad R_e^{\text{FF5}} = r_f + \sum_k \beta_k \Pi_k, \quad (32)$$

with r_f the 10Y Treasury (FRED:DGS10); both are clamped to $[r_f + 2\%, r_f + 12\%]$. Cost of debt is $R_d = r_f + \text{spread}$, where the spread comes from a Damodaran-style synthetic-rating table keyed on interest coverage EBIT/Interest, blended with the realized interest/debt rate when that looks plausible. Then

$$\text{WACC} = \frac{E}{V} R_e + \frac{D}{V} R_d (1 - \tau), \quad V = E + D, \quad (33)$$

E market cap, D total financial debt, clamped to $[6\%, 16\%]$. The headline uses R_e^{CAPM} (the market convention and a stable discount rate); the multi-factor R_e^{FF5} is reported alongside as an exhibit. For the sum-of-the-parts DCF, `segment_wacc` shifts the equity beta and adds a growth-risk premium so higher-growth segments carry a higher discount rate.

9.3 Reverse DCF, scenarios, sum-of-the-parts, triangulation

Reverse DCF solves, by bisection, the flat growth g^* (or steady margin m^*) such that $\text{PerShare}(g^*) = \text{price}$ — i.e. what the market is currently pricing in. *Scenario weighting* runs the DCF once per upside/base/downside scenario (sharing one deterministic share count and net debt) and combines the per-share values V_s with probabilities $\sum_s p_s = 1$ into $\text{FV} = \sum_s p_s V_s$. *Sum-of-the-parts* values each reportable segment on its own growth/margin and a segment-specific WACC, with segment $\text{FCF} \approx \text{NOPAT} \times \text{conversion}$, and sums the segment enterprise values. *Triangulation* builds a football field from the SOTP range (declared primary), the consolidated DCF scenario range, and peer-multiples-implied values, reporting implied upside $\text{FV}/\text{price} - 1$.

10 Scoring and ranking models

10.1 Value + sentiment interest score

Code: `api/routers/screener_agent.py`. Each component is min-max normalized across the shortlist to $[0, 1]$ (v_{PE} inverted — cheaper is better), plus a normalized alpha v_α from the qlib model:

$$\text{score} = w_\alpha v_\alpha + (1 - w_\alpha) \sum_k w_k v_k, \quad \text{interest} = 100 \cdot \text{score}, \quad (34)$$

with $w_\alpha \approx 0.40$ (dominant when the alpha model covers the shortlist). The fundamental sub-weights w_k (FCF-yield, P/E, growth, MD&A tone, news) start from availability-branched defaults and are re-split between the *value* and *growth* blocks by the current regime's factor IC (§8.6): with family emphases $e_{\text{val}}, e_{\text{grw}}$, the fundamental budget is allocated $\propto e_{\text{val}} : e_{\text{grw}}$. With no model or IC available it reduces exactly to the fixed-weight composite.

10.2 Group relative-value composite

Code: `ai_analyst/committee/group.py`. For each metric k (P/E, EV/EBITDA, P/B, FCF yield, dividend, revenue growth/CAGR, gross/operating margin), a cross-sectional z -score $z_{k,i} = (x_{k,i} - \bar{x}_k)/\hat{\sigma}_k$ and a signed weight w_k (negative for cheaper-is-better multiples). Loss-making multiples (negative P/E, EV/EBITDA, P/B) are excluded from their metric's population rather than ranked as “cheap”. The composite is

$$\text{composite}_i = \sum_k w_k z_{k,i}, \quad (35)$$

ordered descending; terciles map to attractive / fair / expensive. The per-name audit trail ($x \rightarrow z \rightarrow \times w \rightarrow \text{contribution}$) sums exactly to the composite. An optional LLM narrative may be attached; the ranking itself is deterministic.

10.3 MD&A management tone

An LLM reads an excerpt of each name's latest MD&A and returns a structured tone in $[-1, 1]$, a positive/neutral/negative guidance label, and risk flags; the tone feeds v_{tone} in §10.1.

Part V How the models connect

11 The model map

The three quantitative paths and the valuation path meet at the investment committee. The valuation path answers what a business is worth; the quant path answers what to expect and how to size it; the macro-cycle path conditions the scoring on the regime; and the committee fuses the deterministic evidence and the quant signals into a reasoned, auditable verdict.

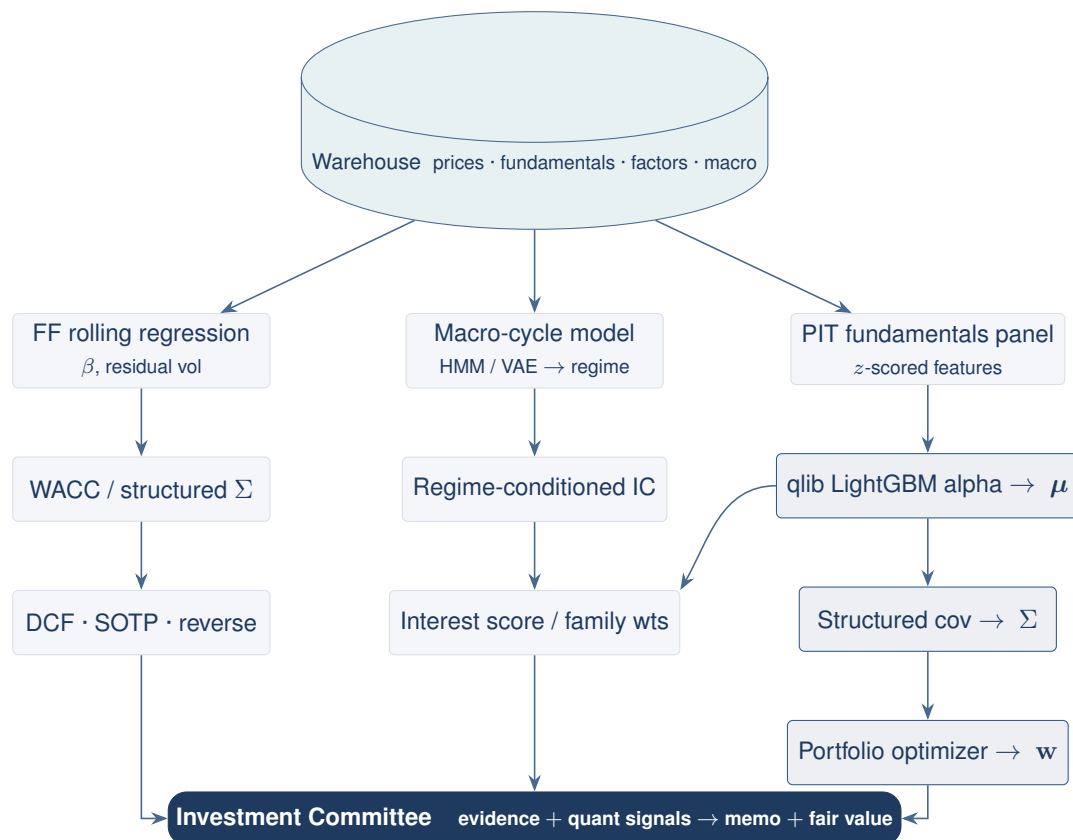


Figure 1. The model map. Statistical, macro, and valuation paths converge on the committee, which produces a reasoned valuation with its evidence rather than a single label.

Independent, systematic research for educational purposes. Estimates are model outputs, not investment advice.